THE TOPOLOGICAL HETEROGENEOUS MANIFOLD ENGINE

(THME)

A Research Specification for Execution-Weighted Layerwise Format Allocation, Adaptive-Depth Routing, and Hybrid Sequence Modeling

Ed Epiphanes

Independent Research | Research Specification and Verification Programme

Version 2.0 | 6 November 2026

DOCUMENT STATUS

Research proposal and experimental protocol. No THME integrated training result or hardware performance result is claimed in this revision. The numeric 80/20 routing scenario and all allocation examples are design hypotheses unless explicitly identified as published results.

Primary objective: make the first experiments cheap, decisive, reproducible, and worth the hardware budget.

Document control and reading guide

Field	Specification
Document purpose	Define a testable THME research programme and a concrete layerwise α/β/γ allocation method.
Version	2.0 - research redesign and expanded validation protocol.
Source basis	THME draft 1.0 (67 pages), supplemented by cited literature verified for this revision.
Empirical status	No new THME training runs or hardware measurements are reported here.
Scope treatment	This revision is a forward-looking research specification. It does not reproduce a historical prototype-defect catalogue.
Primary design clarification	The 80/20 figure describes token routing at a specified exit checkpoint. Each layer independently receives its own binary/ternary/phase channel proportions.
Intended readers	Researchers in efficient language models, quantization, recurrent sequence models, adaptive computation, and inference systems.

How to read this document

Sections 1-4 define the research thesis and model components. Sections 5-8 specify the layerwise allocator and its interaction with the 80/20 routing scenario. Sections 9-13 give training, memory, and hardware accounting. Sections 14-19 specify the experiments, statistical design, compute gates, and stopping rules. Appendices provide a compact algorithm, experiment register, variable glossary, and reproducibility checklist.

Epistemic conventions

Label	Meaning
Established background	A result or method supported by cited prior research; its relevance to THME is still bounded by the cited study.
Design proposal	A specific mechanism proposed for THME but not yet tested in an integrated THME model.
Projection	A calculation based on explicitly stated assumptions; not a measurement.
Open question	A point that requires mathematical specification, empirical comparison, or hardware validation.

Abstract

The Topological Heterogeneous Manifold Engine (THME) is proposed as a heterogeneous language-model architecture that allocates representational formats and computation non-uniformly across model depth. This revision makes a key distinction explicit: a target 80/20 token-routing policy is separate from the channel-format distribution assigned to each layer. For every layer l, the channel width is partitioned into binary, ternary, and phase-oriented families with layer-specific proportions (α_l, β_l, γ_l), where $\alpha_l + \beta_l + \gamma_l = 1$. These proportions are not prescribed by a single depth curve. They are selected by an execution-weighted, sensitivity-aware, hardware-calibrated allocation procedure.

The proposed allocator estimates the probability q_l that a token reaches layer l, measures the quality impact and implementation cost of candidate formats for channel groups within that layer, and solves a constrained discrete assignment problem. Under the illustrative two-path scenario, all tokens traverse an early prefix, 80% exit at a

selected checkpoint, and 20% continue through deeper layers. The resulting execution probabilities change the expected cost of formats in later layers without changing the fact that all model weights must be stored. The optimization therefore separates whole-model storage, expected per-token memory traffic, quality impact, latency, and operator-specific compute.

The revised programme prioritizes four core hypotheses: low-bit weights, hybrid recurrent/attention sequence mixing, adaptive-depth computation, and sensitivity-driven layerwise format allocation. Phase recurrence, consensus-based retrieval, Clifford features, stabilizer-inspired projection, and attractor computation remain optional, isolated hypotheses. The document specifies the formal objective, optimization workflow, calibration strategy, matched baselines, synthetic and language evaluations, energy measurements, statistical controls, compute caps, and stage gates. The central methodological rule is to reject components that do not improve a measured quality-cost frontier before they are allowed into larger training runs.

This is a research proposal. Published results cited below establish precedents for individual techniques; they do not validate THME as a combined architecture.

Executive research brief

The central proposition

THME should be evaluated as an architecture that allocates different computational representations to different layer functions and different token-computation paths. A layer that processes every token may justify a different format mix from a deep layer that processes only a selected subset. A layer with unusually high quantization sensitivity may need more robust channels regardless of its depth. A layer that contributes little to the loss may tolerate a cheaper representation. The allocation should therefore be derived from measured sensitivity and realized cost, not from depth position alone.

The 80/20 scenario is a routing prior, not a format schedule

For an illustrative exit checkpoint at 30% of network depth, the target scenario is: 80% of token positions exit after the checkpoint, while 20% continue to full depth. All tokens still pass through the early prefix. This implies $q_l \approx 1$ for layers in the common prefix and $q_l \approx 0.2$ for layers after the checkpoint, if routing is a single binary decision and the target is achieved exactly. If the router uses several exits, q_l is instead estimated from the empirical fraction of tokens that reach each layer. Neither case specifies α_l, β_l, or γ_l.

The format optimizer chooses a separate distribution for every layer. It may choose a ternary-heavy early layer, a phase-rich deep layer, or a higher-precision exception in any layer if its measured sensitivity and hardware cost justify that choice. No monotonicity with depth is assumed in the primary method.

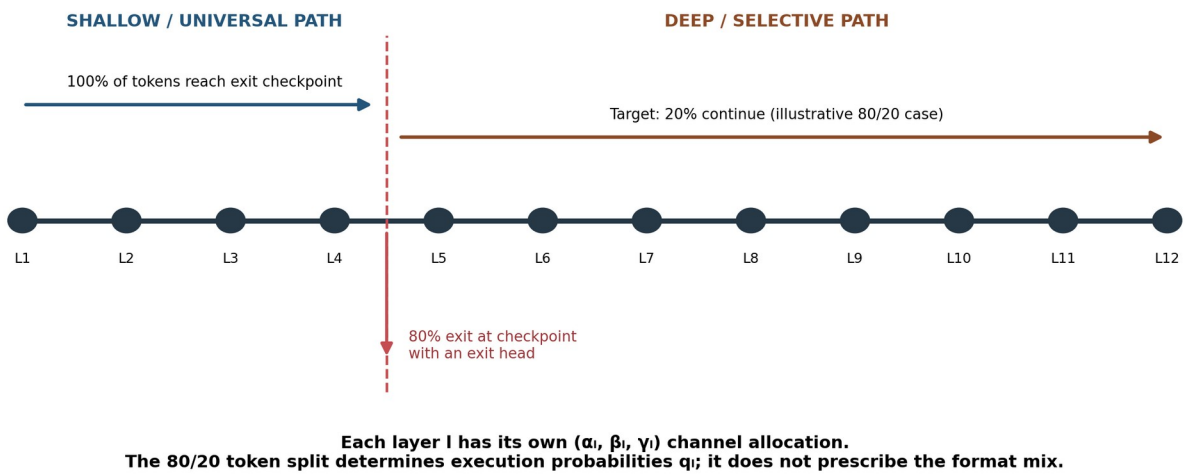


Figure 1. Illustrative token-routing scenario. The per-layer format allocations are independent variables chosen by the allocator; they are not inferred from the 80/20 ratio.

Immediate research priorities

- Implement one sequence mixer first: a strong gated-delta or Mamba-2 family baseline, plus a small attention budget for exact or high-fidelity retrieval.
- Use the layerwise execution-weighted allocator to choose α_l, β_l, and γ_l from measured candidate distortion and hardware costs.
- Treat direct early exit and self-speculative decoding as separate operating modes with different quality guarantees.
- Run zero/low-training-cost operator, calibration, and kernel tests before any full training campaign.
- Only integrate optional geometry, stabilizer, CRM, or attractor components after they beat simpler controls at equal budgets.

1. Research thesis, objectives, and falsifiable claims

1.1 Research thesis

A language model need not use the same numerical representation, sequence mixer, or computation depth at every layer and for every token. If representation families and compute depth are allocated according to (i) the probability that a token reaches a layer, (ii) the layer/group contribution to predictive loss, (iii) the hardware cost of each operator, and (iv) deployment constraints, then the model may improve its quality-energy-memory-latency frontier relative to strong uniform-format and uniform-depth baselines.

1.2 Primary objectives

- Develop and evaluate per-layer channel allocations α_l, β_l, γ_l rather than a single global or sinusoidal schedule.
- Use empirical execution probabilities q_l to weight inference cost and routing-conditioned quality impact.
- Specify the binary, ternary, and phase channel families so that their stored bits, scale factors, states, conversions, and kernels can be counted.
- Identify the cheapest sequence mixer that preserves required recall and language-model quality.
- Quantify whether adaptive depth and speculative verification improve measured energy and latency rather than only estimated FLOPs.
- Create a compute-gated experimental procedure that stops unpromising branches before expensive training.

1.3 Primary hypotheses

ID	Hypothesis	Primary evidence
H1 - Heterogeneous allocation	Layer-specific α_l/β_l/γ_l selected by sensitivity and hardware cost improves quality-cost trade-offs over fixed schedules at equal storage or energy.	Pareto frontier; ablation against uniform, fixed-depth, and manually scheduled allocations.
H2 - Low-bit representation	Ternary and binary channel families reduce realized weight traffic and/or energy while meeting a preregistered quality margin.	Packed-model bytes, kernel traces, held-out NLL, energy/token.
H3 - Sequence mixing	A recurrent/attention hybrid preserves target recall quality with less context-dependent memory than an attention-heavy reference.	MQAR, RULER, long-context tasks, memory scaling, NLL.
H4 - Adaptive depth	A learned per-token router and/or self-speculative decoding reduces measured decode cost while preserving required quality or target distribution.	Exit calibration, acceptance, tokens/s, joules/token, task metrics.
H5 - Joint optimization	Optimizing routing reach probabilities and layerwise format allocation in an alternating offline procedure outperforms optimizing either independently.	Ablation of joint versus routing-only and allocation-only optimization.
H6 - Phase recurrence	A stable complex/phase recurrence provides a useful long-range inductive bias at a net quality-cost advantage over a real-valued recurrence.	Recall and NLL versus phase bit-width, magnitude precision, and actual kernel cost.

1.4 Secondary hypotheses

ID	Component	Decision standard
H7	Consensus-Read Mixer (CRM)	Keep only if independent read-head disagreement predicts errors beyond entropy/margin and adaptive extra reads save cost at equal quality.
H8	Clifford/geometric features	Require a gain on geometry-native tasks first and then language tasks at matched parameters, compute, and representation width.
H9	Stabilizer-inspired projection	Require improved downstream quality and signal retention, not merely lower raw quantization error.
H10	Hopfield/attractor refinement	Require improvement over K ordinary layers or an established retrieval operator at equal FLOPs and memory traffic.
H11	Offline optimization	Require measured improvement from channel allocation, placement, or kernel configuration; do not put general-purpose solvers in the per-token path.

2. Positioning and related research

THME does not claim that ternary weights, recurrent state-space models, early exits, or mixed-precision optimization are novel individually. Its research contribution must be the precise allocation and coordination rule, the resulting architecture, and reproducible evidence that the combined system improves a practical frontier.

2.1 Native low-bit language models

BitNet b1.58 established a serious precedent for training language models with ternary-valued weights and non-standard low-bit inference. The BitNet b1.58 2B4T report describes a 2-billion-parameter model trained on 4 trillion tokens and reports comparisons against open-weight models of similar size [1]. This supports including a native ternary baseline. It does not imply that arbitrary post-training ternarization, mixed binary/phase channel allocations, or an unoptimized kernel will retain quality or efficiency.

2.2 Recurrent and hybrid sequence models

Mamba-2 connects state-space models and attention through structured state-space duality and reports a hardware-aware core layer competitive with Transformers at small and medium scale [2]. Gated Delta Networks combine selective gating with delta-rule memory updates and report improvements over Mamba-2 and DeltaNet on retrieval and language benchmarks [3]. Preconditioned DeltaNet (ICML 2026) adds curvature-aware preconditioning and reports improvements on synthetic recall and language modeling at 340M and 1B scales [4]. These works make a stronger current comparison set than a Transformer-versus-Mamba-only test.

However, efficient recurrent models can still underperform attention on associative recall. Zoology formalizes multi-query associative recall (MQAR) and finds that recall is a major source of the quality gap for some attention-free models; sparse attention hybrids can substantially close that gap [5]. THME must therefore test recall explicitly instead of inferring it from perplexity alone.

2.3 Adaptive compute and self-speculation

Mixture-of-Depths allocates compute to selected sequence positions under a fixed per-layer capacity, making token-level compute non-uniform while keeping tensor shapes predictable [6]. LayerSkip trains for early exit and combines an early layer path with verification by remaining layers; the published work reports workload-dependent speedups and preserves the target distribution through the speculative procedure [7]. These are direct comparators for THME routing. The 80/20 policy should be compared with learned capacity routing and with self-speculative decoding, not treated as inherently optimal.

2.4 Hardware-aware mixed precision

HAQ uses hardware feedback to choose layerwise precision and emphasizes measured latency and energy rather than proxy metrics alone [8]. HAWQ-V3 formulates mixed-precision quantization as an integer optimization problem balancing perturbation with memory and latency constraints [9]. AWQ uses activation statistics to identify salient channels and protect them during quantization [10]; SmoothQuant shows that equivalent transformations can move quantization difficulty between weights and activations [11]. THME should adopt this discipline: allocation must be sensitivity-aware and hardware-aware, and activation effects must be measured alongside weight distortion.

2.5 Evaluation of long context

RULER extends simple needle-in-a-haystack tests with multiple needles, multi-hop tracing, and aggregation tasks, demonstrating that claimed context length does not necessarily equal effective context capability [12]. THME should use RULER or a comparable controlled suite in addition to MQAR and real text evaluation.

3. Architecture overview

3.1 High-level structure

Let the network contain L ordered layers. Each layer consists of a token-mixing operator, a channel transformation, normalization/residual handling, and optional routing or exit logic. The operator may be recurrent, attention-based, or a hybrid. Within each layer, channel groups are assigned to one of three THME representation families: binary (B), ternary (T), or phase-oriented (P). A separate router decides whether each token continues to additional layers.

The high-level design therefore has two different allocation axes:

- Representation allocation across channels within layer l: (α_l, β_l, γ_l).
- Computation allocation across tokens and depth: q_l, the probability that a token executes layer l.

These variables interact through expected cost and quality, but they are not interchangeable. An 80/20 token policy does not mean 80% binary and 20% ternary, nor does it force the same α/β/γ mix in each layer.

3.2 Three execution zones as a convenient reference, not a hard law

Zone	Typical depth role	Token participation in 80/20 example	Allocation principle
Surface / common prefix	Early layers and exit checkpoint	Approximately 100% of tokens	Optimize for high-frequency execution; retain whatever channel families empirical sensitivity requires.
Midwater / selective path	Layers after the exit checkpoint	Approximately 20% of tokens	Can justify richer operators if their quality benefit outweighs their conditional cost.
Abyss / hard-token resolution	Deep layers or refinement blocks	Subset of continuing tokens	Allocate based on conditional sensitivity, not simply because depth is high.

Zones are explanatory. A learned router may create multiple exit depths; in that case q_l is measured rather than defined by a single cutoff.

4. Representation families and operational definitions

A format family must be an executable operator specification, not merely a label. The following definitions are proposed as minimum operational contracts. Exact activation precision, scale layout, packing, and kernels must be recorded for every experimental variant.

4.1 Binary family B

The binary path uses signed weights $w \in \{-1,+1\}$, or another explicitly declared two-state code, with an associated scale factor where required. A bit value alone does not specify the numerical scale, accumulation width, activation representation, or kernel. The experiment must declare whether binary weights are applied to 1-bit, low-bit, or higher-precision activations and how scale compensation is performed.

4.2 Ternary family T

The ternary path uses $w \in \{-1,0,+1\}$. For deployment, the reference physical representation is two-bit packing unless a verified entropy-coded kernel is used. Quantization-aware training should preserve a full-precision master parameter or another optimizer representation as needed, while the inference artifact stores the packed values plus all scales, masks, and metadata. The number 1.585 bits is the ideal information-theoretic entropy for equally likely ternary symbols; it is not the physical storage cost of a two-bit packed implementation.

4.3 Phase family P

The phase family is defined first as a phase-quantized recurrent channel, rather than as a generic “phase weight” with unspecified operations. One candidate diagonal recurrence is:

$$h_t = \lambda \odot h_{t-1} + Bx_t, \qquad \lambda_j = \rho_j \exp(i\theta_j), \qquad 0 < \rho_j < 1$$

$$\theta_j = 2\pi k_j / 2^B, \qquad k_j \in \{0,\ldots,2^B-1\}$$

Here the phase index requires B bits, but the decay magnitude ρ, input/output maps, state precision, and conversion/accumulation operations also require storage and compute. These costs must be explicitly reported. The stable decay condition is $|\lambda_j|=\rho_j<1$. If a separate magnitude code is used, its bit-width is b_ρ and its storage is added to the phase index. The phase-family cost is therefore not equal to B bits per channel unless all other terms are fixed, shared, or otherwise accounted for.

4.4 Cross-family interfaces

Binary, ternary, and phase channels must define compatible residual and normalization interfaces. A layer may concatenate family outputs, project them into a common residual width, or use a structured block map. Interface parameters and conversion operations are part of the cost. No claim of “no type conversion” or “free phase addition” should be made unless the kernel demonstrates it.

4.5 Precision and fallback policy

The primary THME study retains B/T/P as the three allocation families. A higher-precision fallback (BF16, FP16, FP8, or 4-bit) should nevertheless be included as a control and may be used for numerically sensitive tensors such as embeddings, output heads, normalization scales, and selected residual channels. Such exceptions must be counted in both model size and runtime traffic; they must not be hidden outside the model budget.

5. Execution-weighted layerwise format allocation

5.1 Formal definition of α_l, β_l, γ_l

Let layer l have residual/channel width d_l. Divide the width into G_l channel groups, with group g containing n_{lg} channels. Each group is assigned exactly one representation family. Let $z_{lgs} \in \{0,1\}$ indicate assignment of group g in layer l to family $s \in \{B,T,P\}$. Then:

$$\sum_s z_{lgs} = 1 \qquad \text{for every layer } l \text{ and group } g$$

$$\alpha_l = (1/d_l) \sum_g n_{lg} z_{lgB}; \quad \beta_l = (1/d_l) \sum_g n_{lg} z_{lgT}; \quad \gamma_l = (1/d_l) \sum_g n_{lg} z_{lgP}$$

$$\alpha_l + \beta_l + \gamma_l = 1, \qquad \alpha_l, \beta_l, \gamma_l \in [0,1]$$

If all groups are equal-sized, these values are channel fractions. If group sizes differ, use the weighted count above. The deployed model rounds to valid group/channel counts and respects packing alignment. The vector $(\alpha_l,\beta_l,\gamma_l)$ is a distinct decision for each layer; it is not required to vary smoothly or monotonically with depth.

5.2 Token reach probability q_l

Let $I_{tl}=1$ if token position t executes layer l and 0 otherwise. Define:

$$q_l = E[I_{tl}] = \Pr(\text{token } t \text{ reaches layer } l)$$

For a simple two-path policy with exit checkpoint e, where the first e layers are executed by all tokens and a fraction p_continue proceeds thereafter:

$$q_l = 1 \quad \text{for } l \le e; \qquad q_l = p_continue \quad \text{for } l > e$$

For the illustrative p_exit=0.8 case, p_continue=0.2. If the router uses several exits or capacity-limited routing at every layer, q_l becomes the empirical reach rate for that layer. It may vary by token domain, sequence position, batch composition, or task class; where material, use q_l(c) for token class c rather than one global average.

5.3 Why layer-specific allocation is necessary

A fixed depth curve assumes that the best format mix is determined primarily by layer position. That is a hypothesis, not a general law. Actual layer requirements depend on multiple quantities:

- Execution frequency: early layers are read for nearly every generated token; later weights may be accessed only for continuing tokens.
- Loss sensitivity: a layer or channel can be unusually important even if it is shallow or rarely executed.
- Operator role: attention, recurrence, MLP, normalization, and output-projection channels have different numerical and memory behavior.
- Activation distribution: outliers and channel salience are layer- and workload-dependent.
- Kernel realization: a nominally lower-bit representation may be slower if unpacking, conversion, or fallback operations dominate.
- Quality conditionality: deep layers may be critical specifically for difficult tokens, so their sensitivity must be measured on the routed subset as well as on all tokens.

The allocator should therefore be free to choose a different distribution for every layer. A phase-heavy late layer is possible but not mandatory. A sensitive early layer may receive more ternary or phase channels even though it executes for every token. The output allocation is workload- and hardware-profile-specific; one universal α/β/γ schedule should not be assumed to be optimal across CPUs, GPUs, batch sizes, or context lengths.

5.4 Illustrative 80/20 reach profile

Layer group	q_l in two-path example	What q_l means
L1 to L_e	1.00	Every token executes these layers.
L_(e+1) to L	0.20	Only the continuing 20% execute these layers.
Whole model weights	Stored for 100% of layers	Exit routing does not reduce static artifact size by itself; it reduces expected reads/compute for skipped layers.

The values above describe one target scenario only. The achieved q_l vector must be logged after training and during deployment tests. A nominal router threshold is not evidence that the 80/20 split was realized.

5.5 Baseline schedules to compare

The original trigonometric schedule may remain as a fixed-schedule baseline, but it should not be privileged. Compare at least:

- Uniform allocation: same α/β/γ in all layers.
- Original analytic schedule: normalized trigonometric shares evaluated at each layer depth.
- Zone-wise schedule: one mix per zone, chosen on the tuning split.
- Layerwise learned schedule: a differentiable relaxation of per-layer mix logits, then discretized.
- Execution-weighted sensitivity allocator: the primary proposed method.
- Randomized schedule: a negative control with the same global average bits and parameter count.

6. Proposed allocator: execution-weighted heterogeneous allocation (EWHA)

The primary proposed mechanism is named Execution-Weighted Heterogeneous Allocation (EWHA). It is an offline model-design procedure. It does not solve a general optimization problem for every token during inference. Its output is a fixed channel-to-format map (and optional routing thresholds) that can be compiled into a static model artifact.

6.1 Inputs to EWHA

Input	Definition	How to estimate
q_l or q_l(c)	Probability that a token executes layer l, globally or by token class.	Run a calibrated baseline/router over representative prompts; log execution counts by layer and domain.
Ω_lg or ΔL_lgs	Sensitivity of group g in layer l to format s.	Activation-aware statistics, Fisher/Hessian proxy, layer-output reconstruction, or empirical change in held-out NLL under candidate replacement.
b_lgs	Actual stored bits including packed values, scales, zero masks, phase magnitude, metadata, alignment and state where relevant.	Serialize candidate tensors and inspect actual artifact bytes.
e_lgs	Expected energy cost of executing candidate group/operator.	Kernel microbenchmarks plus measured memory traffic and calibrated platform energy.
t_lgs	Latency contribution under specified batch/context conditions.	Real kernel and end-to-end benchmarks; not FLOP-only estimates.
c_lgs	Compute/operation count and auxiliary conversion cost.	Profiler trace or analytically verified kernel accounting.
Constraints	Quality tolerance, model size, memory budget, p95 latency and expected energy limits.	Pre-register deployment envelope.

6.2 Sensitivity estimation

Weight magnitude alone is not a sufficient measure of channel importance. EWHA should combine several inexpensive proxies and verify their ranking empirically. For each layer/group, estimate:

- Activation salience: activation RMS, percentiles, outlier frequency, and input-output covariance on calibration data.
- Gradient/Fisher sensitivity: diagonal empirical Fisher or an appropriate Hessian trace/diagonal approximation where affordable.
- Layer-output reconstruction error: difference between the reference and quantized group outputs on representative hidden states.
- Loss sensitivity: change in held-out token-level NLL when one group or a small set of groups is quantized, with all other layers frozen.
- Conditional sensitivity: repeat the estimate for tokens that exit early and for tokens that continue, where sample counts permit.

The practical first-pass metric should be empirical: temporarily replace a candidate layer/group with a proposed format, evaluate its output or NLL error on a fixed calibration batch, and record the incremental cost. Fisher/Hessian or activation statistics can rank candidates before this expensive step. Do not assume a sensitivity proxy is accurate until it predicts measured perturbation.

6.3 Discrete candidate characterization

For every group g in layer l and candidate family s, construct a measurement record. The record contains quantization distortion, task-loss delta, physical bytes, kernel latency, energy, and any conversion or residual cost. For phase groups, separately record phase-index bits, magnitude/decay bits, state bytes, Cartesian conversion, and accumulation. For binary/ternary groups, separately record activation precision, scale storage, zero handling, and accumulator width.

6.4 Core constrained optimization

The preferred formulation is a multiple-choice integer optimization over channel groups. Each group selects exactly one candidate family. A simplified additive screening objective is:

$$\text{minimize_z} \quad \Sigma_l\, q_l\, \Sigma g\, \Sigma_s\, z_{lgs} \cdot E_{lgs}$$

$$\text{subject to} \quad \Sigma_l\, q_l\, \Sigma g\, \Sigma_s\, z_{lgs} \cdot \Delta L_{lgs} \le \varepsilon_Q$$

$$\Sigma_l\, \Sigma g\, \Sigma_s\, z_{lgs} \cdot B_{lgs} \le S_max$$

$$\Sigma_l\, q_l\, \Sigma g\, \Sigma_s\, z_{lgs} \cdot R_{lgs} \le R_max$$

$$\Sigma_l\, \Sigma g\, \Sigma_s\, z_{lgs} \cdot T_{lgs} \le T_budget$$

$$\Sigma_s\, z_{lgs} = 1; \qquad z_{lgs} \in \{0,1\}$$

Here E_lgs is measured or calibrated energy per token conditional on executing the group; ΔL_lgs is the conditional quality penalty for tokens that execute layer l; B_lgs is stored bytes; R_lgs is conditional memory traffic; and T_lgs is the conditional latency/compute contribution. The q_l multiplier converts conditional energy, loss impact, traffic and layer cost into expected per-token quantities. Storage is not multiplied by q_l because all weights remain part of the deployed artifact. The objective may instead minimize expected bytes or latency under an energy cap. The result should be reported as a Pareto set across multiple budgets rather than only one scalarized optimum.

This simplified model assumes group costs and quality penalties are approximately additive. That assumption is used only for screening. Quantization interactions, kernel fusion, memory coalescing, activation changes, and routing feedback can make the true objective non-additive. Every candidate selected by EWHA must be validated as a complete model after fine-tuning and with end-to-end measurements.

6.5 Why a simple layer-level LP is not enough

If the optimizer assigns one continuous mix to each layer and uses a linear distortion objective with only one global resource constraint, the optimum can be a sparse “bang-bang” solution: most layers become pure format choices, with very few mixed layers. That is not necessarily the desired THME behavior. Use channel groups as allocation units, use empirically measured discrete candidates, and include realistic storage/latency constraints. If smooth internal mixtures are beneficial, model diminishing returns or group-level marginal distortion rather than forcing a linear layer-level approximation.

6.6 Hardware-aware objective and non-additive effects

Hardware cost is not a property of bit-width alone. It depends on the target processor, kernel implementation, tensor layout, group size, batch size, sequence length, and whether the weights are resident in cache or repeatedly fetched from DRAM. A two-bit packed layer can lose its advantage if unpacking or unsupported operations cause conversion to FP16. EWHA must use target-hardware measurements and include a whole-layer kernel cost model. For final selection, benchmark candidate bundles or complete models, not only per-channel arithmetic estimates.

6.7 EWHA algorithm

1. Profile a high-quality reference model on representative prompts and tasks. Record q_l, per-layer compute, bytes, activation distributions, and exit decisions.
2. Partition each layer into allocation groups. Choose group size based on the smallest unit supported efficiently by the target packing and kernels.
3. For each group, generate binary, ternary, and phase candidates with complete scale/state/interface definitions.
4. Estimate sensitivity using activation statistics and a low-cost reconstruction or NLL perturbation pass. Retain a small candidate set per group.
5. Microbenchmark each candidate and record real storage, latency, throughput, and energy. Reject candidates without a viable implementation.
6. Solve the discrete allocation problem across a grid of quality and resource budgets. Generate several Pareto candidates.

7. Fine-tune or distill only the top candidates. Re-profile q_l because quantization and early-exit training can change routing behavior.
8. Re-estimate sensitivity and repeat optimization only if the measured selection-set improvement justifies another round.
9. Freeze the allocation, export packed weights, and evaluate exactly once on an untouched final test set.

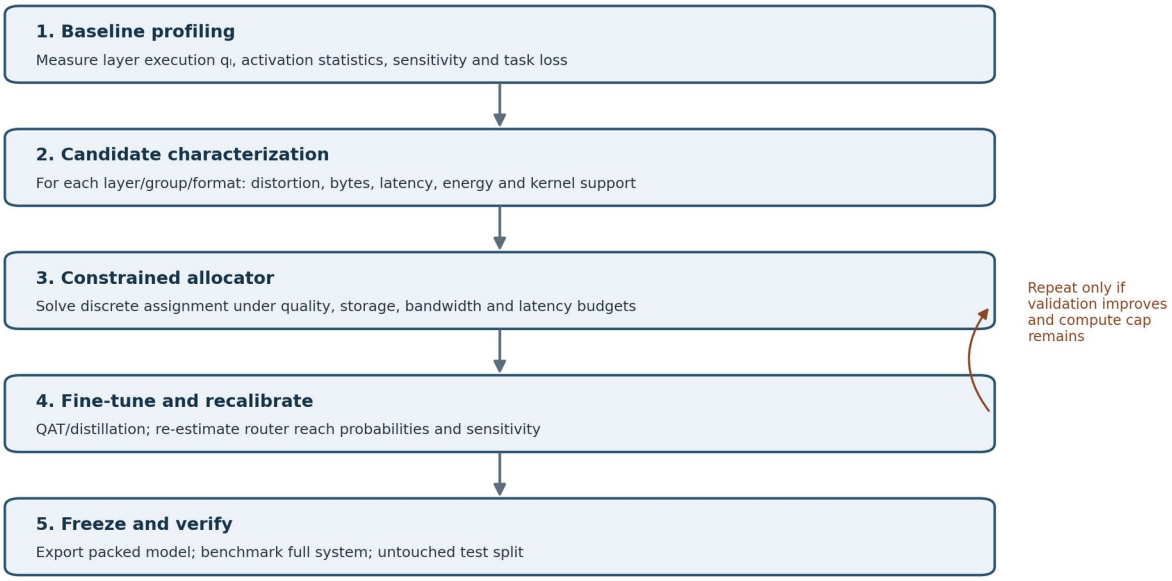


Figure 2. EWHA workflow. The optimization is performed offline; only the frozen allocation and small runtime routing policy are deployed.

6.8 Differentiable alternative

A complementary research path assigns each layer trainable logits u_lB, u_lT, u_lP and uses a softmax relaxation to estimate continuous shares during training. A temperature schedule can sharpen the assignment, after which channel groups are projected onto a valid discrete allocation under the storage budget. This route can capture interactions missed by one-shot sensitivity estimates, but it risks adding substantial training compute and can learn a soft mix that cannot be realized efficiently by the target kernels. It should be evaluated after the discrete offline allocator, not before it.

6.9 Alternating joint optimization of router and format map

The routing policy changes q_l, and q_l changes the expected value of deep-layer precision. EWHA should therefore use a limited alternating procedure: (i) fit or calibrate the router and estimate q_l; (ii) optimize the layerwise format map using those q_l values; (iii) fine-tune the quantized model and router; (iv) recompute q_l and sensitivities; (v) accept the update only if the selection-set quality-cost objective improves. Cap the number of rounds and freeze the final artifact. This is an offline bilevel search, not an unconstrained online self-modifying model.

7. Adaptive-depth routing and the 80/20 scenario

7.1 Definition of the target scenario

Let e be an early-exit checkpoint at approximately 30% of model depth. The initial routing hypothesis is p_exit≈0.8 and p_continue≈0.2. All tokens run the common prefix; tokens that meet the exit policy receive an output from the early head, while tokens that fail the exit criterion continue through deeper layers. The achieved proportions are measured across held-out prompts and reported by domain, sequence position, and token class.

7.2 Expected executed depth

$$\bar{f} = (1/L)\,\Sigma_l\, q_l, \qquad \text{for equal-cost layers}$$

$$\bar{f} = 0.8 \times 0.3 + 0.2 \times 1.0 = 0.44, \qquad \text{for the idealized two-path 80/20 example}$$

The 0.44 figure is an expected depth fraction, not an 80% compute saving. For unequal layer costs c_l, use:

$$C_{expected} = \Sigma_l\, q_l\, c_l$$

The complete latency model must also include exit-head cost, routing cost, gather/scatter or capacity management, speculative drafting and verification, recurrent-state maintenance, and any extra memory traffic. Wall-clock speedup is not generally the reciprocal of expected depth fraction.

7.3 Router signals

Candidate router features include early-head entropy, top-two logit margin, disagreement between distinct exit heads, hidden-state statistics, and learned confidence features. The realized next-token surprisal is unavailable before the token is emitted and must not be used as a routing input for that prediction. The router must use only information available from the observed prefix.

7.4 Training and calibration

Train the exit head with an auxiliary token-level objective and/or distillation from the full-depth model. The router can be trained with a budget penalty, a capacity constraint, or a dual variable that targets a mean continuation rate. Calibration is essential: threshold selection should be performed on a tuning split, then locked before final evaluation. Report exit rate, quality by exit depth, calibration error, and the relation between router score and downstream token loss.

7.5 Direct exit versus self-speculative decoding

Mode	Mechanism	Quality statement	Primary measure
Direct early exit	Emit from an early head without full-depth verification.	Can alter the output distribution and may reduce quality.	Quality-compute curve, exit calibration, energy/token.
Self-speculative decoding	Use early layers to draft; use remaining layers to verify/correct with a valid speculative algorithm.	Can preserve the target model distribution under the standard acceptance/rejection procedure.	Acceptance rate, tokens per verification, tokens/s, joules/token.

The two modes must not be merged into one result. LayerSkip is a relevant baseline for self-speculation and shows that the best exit layer can depend on model and workload [7].

7.6 Routing controls

- No early exit: all tokens execute all layers.
- Fixed 80/20 router: deterministic target capacity with a calibrated confidence rule.
- Learned per-token router with a compute-budget penalty.
- Mixture-of-Depths-style fixed per-layer top-k capacity routing.

- Self-speculative decoding from the early path.
- Random routing with the same mean compute budget as a negative control.

8. Sequence mixing and memory design

8.1 Required reference configurations

The token-mixing operator should be selected before the exotic representation components. At minimum compare: a causal Transformer with grouped-query attention; Mamba-2; Gated DeltaNet; Preconditioned DeltaNet when reproducible code and compatible kernels are available; and a hybrid recurrent/attention stack. The original THME recurrent layer must be specified as an actual operator with stable update, state dimension, state precision, parallel training algorithm, and causal inference semantics.

8.2 Hybrid attention placement

A hybrid may place sparse or sliding-window attention at selected layers while using recurrence elsewhere. Attention layers preserve a mechanism for content-addressable retrieval; recurrent layers reduce context-growing cache requirements. If any full-attention layers retain all historical keys and values, the overall model still has a cache that grows with context length. A strict bounded-memory claim requires a fixed window, bounded retrieval store, compression/eviction policy, or an architecture with no unbounded historical cache.

8.3 Recall-aware task design

Use multi-query associative recall, key-value overwrite, copy, induction-style tasks, and multi-needle retrieval. Test increasing sequence lengths and the number of stored associations. MQAR is important because recurrent models can appear strong on simple synthetic recall while failing a more realistic multi-query setup [5]. RULER supplies long-context tasks beyond one needle [12].

8.4 State budget

For each recurrent layer, report exact state dimensions, precision, read/write operations, state reset policy, and bytes per active sequence. State memory may be constant in sequence length but can scale with number of layers, heads, hidden state dimensions, and concurrent sequences. The total memory budget must include recurrent states and any attention cache together.

9. Phase, consensus, geometric, stabilizer, and attractor extensions

These mechanisms are retained as optional research branches. None should be part of the first integrated model unless a low-cost isolated experiment shows a benefit.

9.1 Phase-quantized recurrence

Compare real diagonal recurrence, complex diagonal recurrence with full-precision phase, and phase quantization at 4, 6, and 8 bits. Keep the magnitude/decay precision fixed for the first comparison. Then vary magnitude precision separately. Measure phase reconstruction error, hidden-state drift across long sequences, MQAR/RULER accuracy, language loss, state size, and actual kernel cost. The phase path is worthwhile only if a discrete phase variable contributes a useful representation or hardware advantage beyond ordinary rotary embeddings or real-valued recurrence.

9.2 Consensus-Read Mixer

Treat CRM as an independent optional retrieval operator. First test whether multiple independently parameterized read heads are diverse enough to provide error-predictive disagreement. Compare their disagreement against predictive entropy and top-two logit margin. Dithered multiple reads should only be used if the hardware or measured read-noise distribution makes repeated sampling necessary; repeated stochastic reads are not assumed

to be a cheaper substitute for deterministic precision. Test mean, median, trimmed mean, and learned consensus. Adaptive extra reads must demonstrate lower mean reads per token at equal quality.

9.3 Clifford/geometric feature subspace

If investigated, begin with a low-dimensional algebra and a task with a genuine geometric inductive bias (for example, N-body dynamics or point-cloud transformations). Compare against equal-parameter dense, orthogonal, complex, or quaternion layers. For language tasks, test only after a geometry-native advantage is demonstrated. Any grade-truncated product must be specified as an explicit projected operation; do not assume closure or associativity after truncation.

9.4 Stabilizer-inspired projection

Test the projection as a learned or structured subspace constraint, not as established topological error correction. Measure quantization error, task loss, signal retention, and gradient stability. Controls should include RMSNorm, random orthogonal projections, Hadamard rotation, and learned low-rank projections. A decrease in Euclidean quantization error alone is not a sufficient pass condition.

9.5 Hopfield/attractor refinement

A modern Hopfield retrieval block can be tested as a small associative-memory module. Its memory matrix, number of slots, precision, number of iterative reads, and per-token bytes must be charged to the cost budget. Compare K attractor iterations against K ordinary layers and one established retrieval baseline at equal FLOPs and memory traffic. It must show an incremental benefit on multi-hop lookup or controlled reasoning tasks, not merely a change in latent-state norm.

9.6 Causal geometry

The causal cone remains an optional inductive-bias experiment. In an autoregressive language model, position-based causal masking or causal recurrent updates already enforce temporal direction. A geometric constraint must demonstrate a benefit beyond that baseline, such as improved structured temporal generalization. It must not be described as replacing causal masking unless the operator-level proof and tests establish that property.

10. Training and conversion programme

10.1 Two distinct training questions

Training a new model with heterogeneous formats and converting a pre-trained teacher are different experiments. Report them separately. A from-scratch pilot tests whether the architecture can learn under the proposed constraints. A conversion pilot tests whether a known teacher function can be retained under the representation and sequence-mixer changes.

10.2 Minimal conversion order

10. Freeze the teacher and establish reference logits, hidden states, and held-out evaluation.
11. Convert selected MLP/channel groups to ternary or binary while retaining the original token mixer.
12. Apply QAT or a controlled distillation budget; test the EWHA assignment against uniform ternary and 4-bit PTQ.
13. Replace one attention block at a time with the candidate recurrent mixer and distill that block output.
14. Only after the block-level result passes should the conversion be extended to multiple blocks or the full model.
15. Introduce early-exit training after the student has a stable full-depth quality baseline; otherwise routing effects are confounded with conversion loss.

10.3 Distillation objective

$$L = L_{KD} + \mu L_{hidden} + \lambda_{budget} L_{compute} + \lambda_{alloc} L_{format}$$

L_KD is temperature-scaled teacher-student distribution matching; L_hidden is a clearly defined layer-correspondence loss; L_compute penalizes expected executed cost; L_format regularizes or supervises the format allocation where used. The precise normalization, token masking, and teacher/student layer mapping must be stated. Do not apply hidden-state matching to layers not executed by a token unless the relevant teacher target is explicitly constructed.

10.4 Training compute accounting

For a dense Transformer, the common approximation $F_{train} \approx 6ND$ (N parameters, D training tokens) is useful for an initial budget, but it is not a universal estimator for hybrid/recurrent/conditional models. Count each forward/backward operator, teacher forward pass, auxiliary loss, checkpointing, and routing path. Use profiler-derived measured FLOPs or operation counts for the actual architecture. Record sustained throughput rather than peak theoretical FLOPs.

10.5 Training-memory accounting

Report model weights, master weights, optimizer states, gradients, activations, recurrent states, KV cache, temporary dequantization buffers, and optimizer offload separately. Any memory-saving claim must include the actual optimizer and activation configuration. Gradient checkpointing, reversible layers, low-bit optimizer states, frozen embeddings, and LoRA should be treated as controls before proposing a new learning rule solely to reduce training memory.

11. Resource model and deployment accounting

11.1 Distinguish storage, traffic, compute, and latency

Model size is the total serialized representation. Expected per-token weight traffic is the sum of layer bytes weighted by the probability of executing each layer, subject to cache residency and actual memory behavior. Compute cost depends on the operator and activation format. Latency depends on kernel launches, synchronization, batching, occupancy, memory bandwidth, and sequential dependencies. These quantities must not be collapsed into one “efficiency” number.

11.2 Expected weight traffic

$$W_{bytes/token} \approx \Sigma_l q_l \cdot W_{bytes,l,active}$$

$W_{bytes,l,active}$ includes all deployed parameters fetched when layer l runs: packed weights, scales, zero metadata, phase magnitude/indices, biases, and any per-layer auxiliary parameters. It must be measured or derived from the serialized tensor layout. The model artifact size remains $\Sigma_l W_{bytes,l}$ plus non-layer components, regardless of the exit probabilities.

11.3 Expected energy

$$E_{token} = E_{weights} + E_{activations} + E_{state} + E_{KV} + E_{compute} + E_{router} + E_{verify} + E_{system}$$

For a simple DRAM-bound estimate, $E_{weights}$ can be approximated from transferred bits multiplied by a measured energy-per-bit calibration for the actual memory system. That estimate is a first-order model only. It must be checked against the external system meter and profiler counters. Never import an energy-per-operation number from a different process node or hardware generation and present it as a measurement of the target device.

11.4 Full accounting checklist

- Embeddings, output head, normalization, biases, quantization scales, codebooks and alignment padding.
- Recurrent state, attention KV cache, retrieval memory, speculative draft/verification cache and any duplicated state.

- Unpacking, permutation, transposition, format conversion, Cartesian phase conversion and accumulation.
- Kernel launch, routing, gather/scatter, padding, load balancing and synchronization overhead.
- Prefill, decode, batch size, context length, concurrency, prompt/output length and warm-up.
- External wall power or board power with idle baseline, thermal and clock conditions.

11.5 Minimum systems measurements

Metric	Required condition
Tokens/s	Report prefill and decode separately; include batch=1 and at least one higher-concurrency case.
Latency	Median and p95 per generated token; report TTFT separately.
Energy	Joules per generated token at stated context/output lengths; external measurement preferred.
Memory	Peak host RAM, VRAM, model artifact, recurrent state, KV cache, temporary buffers.
Traffic	DRAM/HBM bytes read/written where counters are available; distinguish cache hits.
Quality	Held-out NLL/perplexity and individual benchmark results for the same checkpoint.
Execution	Layer reach histogram, exit rates, average executed depth/cost, speculative acceptance.

12. Experimental baselines and datasets

12.1 Core architecture baselines

Baseline	Why it is required	Comparison control
BF16/FP16 causal Transformer + GQA	Quality reference and robust dense baseline.	Same data, tokenizer, parameter count, context and training budget.
4-bit AWQ or GPTQ Transformer	Strong practical PTQ memory baseline.	Use optimized, supported kernels and report artifact bytes.
Native BitNet-style ternary model	Closest established low-bit baseline.	Match parameter count and training tokens as closely as practical.
Mamba-2	Modern selective state-space baseline.	Official or validated implementation and kernel path.
Gated DeltaNet	Strong gated-delta recurrent baseline.	Use supported optimized implementation.
Preconditioned DeltaNet	Current 2026 recurrent comparator.	Reproduce on compatible scale and hardware; record implementation version.
Hybrid recurrent + sliding-window/sparse attention	Tests the realistic memory/recall compromise.	Match attention budget and total parameters.
LayerSkip self-speculation	Early-exit verification baseline.	Compare on the same base model and task; record draft/verify settings.

12.2 Language and recall evaluation

Evaluation family	Examples	Purpose
Language modeling	Held-out cross-entropy and perplexity on a documented corpus.	Primary quality measure; report per-domain where possible.
Associative recall	MQAR, key-value overwrite, copy, induction-style tasks.	Test retrieval and content-based memory.
Long context	RULER tasks, multi-needle, multi-hop	Test effective context quality across

Evaluation family	Examples	Purpose
	tracing, aggregation.	lengths and complexity.
Reasoning / knowledge	A carefully selected, versioned suite such as MMLU-style, GSM8K-style, coding or instruction following.	Check that NLL preservation does not hide task regressions.
Robustness	Position shifts, distractors, repeated keys, changed write order, long sequences.	Find brittle routing and memory failure modes.

Benchmark data must be decontaminated against training and tuning data to the extent feasible. The final test set should remain untouched until all allocation, router threshold, and checkpoint choices are frozen.

13. Staged validation programme and compute gates

The programme is designed as a funnel. Early stages use algebraic checks, frozen checkpoints, calibration data, synthetic sequences, and kernel microbenchmarks. Expensive training occurs only for mechanisms that survive cheaper falsification tests.

13.1 Gate 0 - specification and cost audit

Work item	Method	Exit criterion
Operator contracts	Write exact tensor shapes, state updates, data types, packing, scales, residual interfaces and causality.	Every candidate has an unambiguous forward and backward specification.
Storage accounting	Serialize representative tensors and inspect actual artifact bytes.	No missing scales, states, metadata or padding in the estimate.
Kernel feasibility	Microbenchmark native kernels or a credible implementation path for each family.	Candidate has a viable target-hardware implementation or is deferred.
Router contract	Define available prefix features, exit head, capacity and acceptance procedure.	No future-token leakage; route decisions are reproducible.

Compute budget: no full model training. Limit work to mathematical tests, small synthetic tensors, serialized artifacts, and kernel microbenchmarks.

13.2 Gate 1 - zero/low-training-cost component screens

- Quantization screen: compare format candidates on frozen teacher activations and layer-output reconstruction. Use 1-5 million representative calibration tokens as an initial configurable range, not a fixed universal requirement.
- Sensitivity screen: rank groups by activation, Fisher/Hessian proxy, and empirical NLL perturbation; test whether the ranking predicts loss impact.
- Recurrent screen: evaluate operator on MQAR, overwrite, copy, and sequence-length extrapolation before language pretraining.
- Router screen: evaluate early-head agreement and calibration on frozen checkpoints; compare entropy, margin, and learned gate.
- Kernel screen: profile complete layer operators and record conversion/unpack costs.

Gate 1 exit: discard any candidate with no plausible quality or hardware advantage, or whose kernel cost negates its nominal bit-width advantage. Only top candidates proceed.

13.3 Gate 2 - minimal model pilot

Train or adapt a 20M-50M parameter pilot only after the Gate 1 candidates are narrowed. Use matched tokenizer, data mixture, context, training tokens, optimizer, and tuning effort. A screening budget of 50M-200M tokens may be considered, then adjusted from a pilot learning curve and available compute. This range is a proposed starting

envelope, not a claim of sufficient convergence. Run a small number of candidates and at least three seeds for screening; reserve five or more seeds for finalist claims.

Variant	Purpose
A. Dense reference	Establish baseline loss and compute.
B. Uniform ternary	Isolate low-bit representation effect.
C. Fixed original schedule	Test the original depth-dependent allocation.
D. Layerwise EWHA	Test the proposed allocation mechanism.
E. EWHA + 80/20 direct exit	Test routing quality-cost trade-off.
F. EWHA + self-speculation	Test distribution-preserving acceleration.
G. Recurrent/attention hybrid	Test sequence mixer independent of allocation.
H. Best components combined	Estimate interaction after isolated effects are known.

13.4 Gate 3 - matched-scale confirmation

Promote only one or two candidates to a larger model or teacher conversion. Test at least two parameter scales if compute permits. Confirm the result over independent seeds and a separate test corpus. Measure actual memory and energy, not only estimated cost. The candidate must outperform the strongest matched baseline on a pre-registered Pareto objective.

13.5 Gate 4 - systems deployment test

Export and freeze the artifact; benchmark the actual deployed kernels. Test the intended CPU/GPU or accelerator path, at batch=1 and a higher concurrency. Verify that no hidden dequantization, fallback, or duplicated state eliminates the predicted savings. Reconcile predicted and measured cost components.

13.6 Gate 5 - scale and independent replication

Only after the core result survives Gate 4 should the project scale toward 1B parameters or beyond. Publish the training recipe, dataset manifest, model configuration, allocation map, routing policy, hardware details, raw measurements, and analysis code so another group can reproduce the result.

14. Statistical design and protection against false discoveries

14.1 Preregistered primary endpoint

Choose one primary quality endpoint and one primary systems endpoint before running the study. A defensible pair is held-out token-level cross-entropy (with a fixed non-inferiority margin) and measured joules per generated token under a specified decode workload. Latency and memory are co-primary deployment constraints or separately preregistered secondary endpoints.

14.2 Seeds, paired examples, and confidence intervals

- Use at least three seeds for screening; use at least five independent seeds for a finalist claim, subject to a power calculation.
- Use the same held-out examples for paired baseline/candidate evaluation. Bootstrap at the sequence/example level, not individual token positions alone, when dependence matters.
- Report mean, standard deviation, 95% confidence interval, and all seed-level results. Do not report only the best checkpoint or best seed.
- Use paired statistical tests where the design supports them. Correct the family of primary comparisons using Holm or another prespecified method.
- Keep tuning, selection, and final test splits disjoint. Touch the final split once after the allocation and router are frozen.

14.3 Non-inferiority and Pareto criteria

Set quality non-inferiority margins before training. A practical initial target might be no more than a 1-2% relative degradation in held-out cross-entropy versus the strongest matched baseline, but the appropriate margin depends on baseline variance and the application. The threshold must not be selected after seeing results. Require a meaningful measured systems improvement beyond meter/profiler noise. Report the full quality-energy-memory-latency frontier.

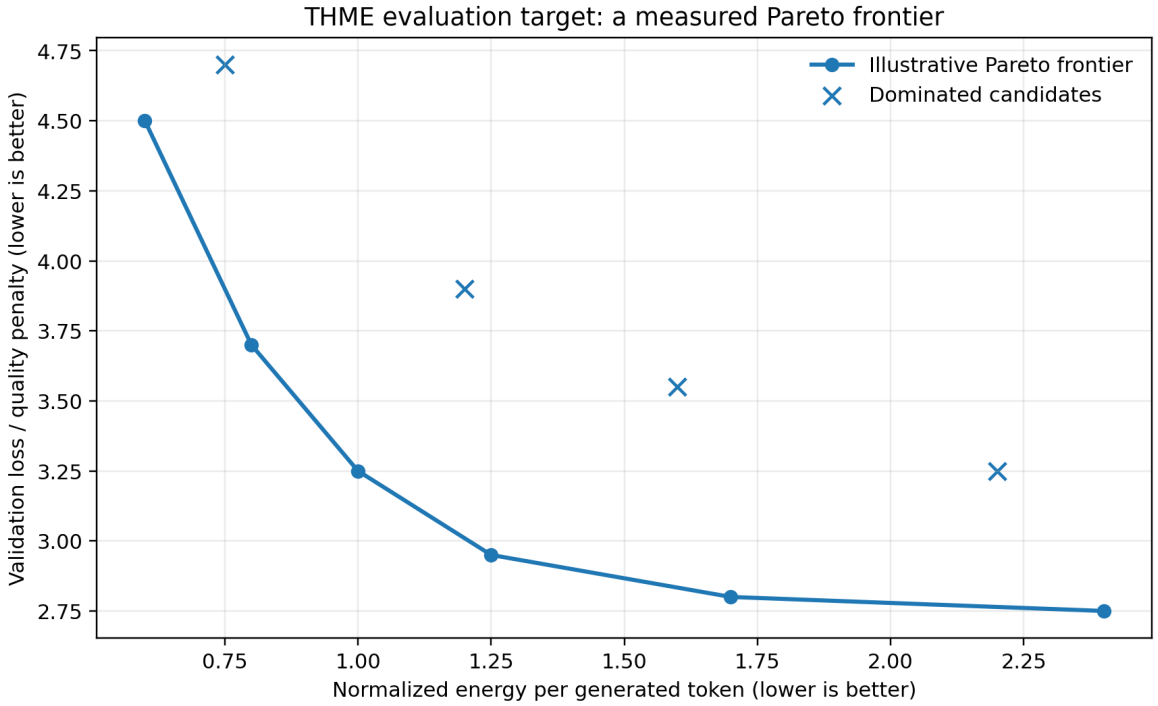


Figure 3. Conceptual Pareto illustration only. No plotted point is an experimental THME result.

14.4 Negative controls

- Random allocation with the same average bits as EWHA.
- Random routing with the same mean number of executed layers.
- Equal-parameter dense layer replacing every proposed geometric or phase operator.
- Equal-FLOP ordinary-layer expansion replacing attractor iteration.
- Unoptimized and optimized kernel paths to expose implementation dependence.
- Layerwise sensitivity ranking shuffled across layers to test whether the measured ranking matters.

15. Compute budget, resource discipline, and kill rules

15.1 Compute should be committed by evidence tier

The study should reserve only a small fraction of total compute for exploratory work. An initial allocation policy is: Gate 0 and kernel screens consume no full training budget; Gate 1 uses frozen checkpoints and small synthetic workloads; Gate 2 receives a capped pilot budget; Gate 3 is reserved for at most one or two finalists; Gate 4 measures deployment; Gate 5 is contingent on a result worth independently reproducing.

Stage	Suggested share of planned compute	Rule
Gate 0	≤1%	No full training.
Gate 1	≤5%	Frozen checkpoint, synthetic, calibration and kernel tests.

Stage	Suggested share of planned compute	Rule
Gate 2	≤15%	Small pilot only; stop weak variants early.
Gate 3	≤35%	One or two finalists; preregistered seeds and matched budget.
Gate 4	≤10%	Systems verification and repeat measurements.
Gate 5	Remaining budget by approval	Scale only after a replicated Pareto gain.

Percentages are a planning envelope, not a mandatory universal allocation. The research lead should revise them to the actual hardware and compute budget before the first run.

15.2 Early stopping rules

- Stop a component after two independent screening rounds show no quality improvement and no measurable hardware advantage over its simplest control.
- Stop a candidate if its best possible measured kernel cost cannot meet the deployment envelope, even before full training.
- Stop a router configuration if it meets the nominal 80/20 rate only by violating the preregistered quality margin or by severe domain-specific failure.
- Stop adding optional components when the core architecture has not produced a Pareto improvement.
- After three complete optimization cycles without a Pareto improvement over the strongest baseline, freeze the negative result and reassess the central hypothesis.

15.3 Compute ledger

For every run, log model size, sequence length, tokens, batch size, training FLOPs or measured GPU-hours, power, memory peak, teacher compute, evaluation compute, seed, checkpoint selection rule, and reason for continuation or termination. The compute ledger is a first-class research artifact, not an administrative afterthought.

16. Expected outcomes and interpretation rules

Observed result	Interpretation
EWHA beats fixed schedules at equal quality and lower measured energy.	Evidence for layerwise sensitivity-/execution-aware allocation.
EWHA improves quality at equal stored bytes but not energy.	Representation allocation may be useful; hardware realization remains unresolved.
Ternary wins on artifact size but not latency/energy.	Compression is established; inference acceleration is not.
Hybrid mixer retains NLL but fails MQAR/RULER.	Recall bottleneck remains; increase sparse attention/retrieval or narrow the claim.
Direct exit saves compute but reduces quality.	Valid quality-compute trade-off; not a lossless speedup.
Self-speculation improves tokens/s but not joules/token.	Latency benefit only; do not claim energy improvement.
Phase or Clifford helps only on its native synthetic task.	Retain as a task-specific operator, not a general LLM component.
Optional components add cost without moving the frontier.	Remove them from the integrated architecture.
No component beats a strong baseline.	A valid negative result; retain the harness and revise or stop the architecture thesis.

17. Risks, confounds, and mitigation

Risk/confound	Why it matters	Mitigation
Unfair baseline tuning	A weak baseline can create a false win.	Equal tuning budget; compare to mature optimized implementations.

Risk/confound	Why it matters	Mitigation
Quality metric masking	Mean benchmark score hides severe task regressions.	Report per-task metrics and token-level NLL.
Router distribution shift	q_l changes by prompt/domain and after quantization.	Measure q_l by domain; recalibrate after fine-tuning.
Sensitivity proxy mismatch	Fisher/activation score may not predict actual loss.	Validate ranking with perturbation NLL and end-to-end ablations.
Kernel conversion overhead	Nominal bit-width may not translate to faster kernels.	Profile fused kernels and include unpack/format conversion.
Memory residency	Layer reads may be served from cache rather than DRAM.	Report cache state and measured memory traffic.
Training/inference mismatch	STE or fake quantization may differ from packed execution.	Compare fake-quant checkpoint against exported packed runtime.
Multiple comparisons	Many allocation schedules increase false discoveries.	Preregister, correct statistics, preserve untouched test split.
Long-context benchmark overfitting	Synthetic recall alone can be gamed.	Use diverse task families and realistic text retrieval.
Hardware dependence	A format optimal on one GPU may be poor on CPU or vice versa.	Declare target platform; report a separate Pareto frontier per platform.

18. Researcher implementation package

A research group should be able to begin with the following artifacts, without first implementing the complete architecture:

- Configuration schema: layer count, group size, operator type, candidate family, bit-width, scale policy, phase magnitude format, activation format, router checkpoint, target hardware.
- Allocation profiler: logs per-layer q_l, token classes, activation statistics, sensitivity estimates, and candidate perturbation effects.
- Kernel benchmark suite: packed and unpacked binary/ternary paths, phase recurrence, accumulation, conversion, and complete layer timing.
- EWHA solver: discrete channel-group assignment with quality, storage, traffic, and latency constraints; exportable JSON/YAML allocation map.
- Routing evaluator: exit-depth histogram, calibration, p_exit, p95 layer reach, direct-exit quality and self-speculative acceptance.
- Evaluation harness: NLL/perplexity, MQAR, overwrite, RULER, downstream tasks, confidence intervals, and benchmark versioning.
- Energy harness: idle baseline, warm-up, external meter capture, device counters, tokens/s, joules/token, context length and concurrency.
- Run registry: dataset checksums, code commit, environment/container, GPU model/driver, seeds, configs, checkpoints, compute ledger and negative results.

19. Recommended first research cycle

The first cycle should be intentionally narrow. It should establish whether EWHA is useful before trying to establish every THME component.

Experiment A - execution-weighted allocation simulation

Take a frozen medium-sized model. Measure q_l for a no-exit baseline and the target 80/20 policy. Characterize binary, ternary, and one phase-recurrence candidate on selected channel groups. Build a measured distortion/cost table. Solve the allocation problem for several quality and storage budgets. Compare with uniform, original

trigonometric, zone-wise, and shuffled-sensitivity schedules. This stage can test whether the allocator produces a materially different and more efficient allocation without a full training run.

Experiment B - export feasibility

Export the top two EWHA configurations into actual packed inference artifacts. Benchmark each selected layer and the complete model path. If the best predicted configuration does not produce a measurable hardware gain after packing and conversion, stop the relevant format branch.

Experiment C - minimal language-model pilot

Only after A and B pass, train or fine-tune the smallest model that can meaningfully measure language quality. Compare dense, uniform ternary, fixed schedule, and EWHA. Use identical data, tokens, tokenizer, and optimization budget. Start with the full-depth model before introducing the 80/20 router.

Experiment D - router integration

Add the exit head and train a router. Test the 80/20 target across several checkpoints. Then evaluate direct exit and self-speculative decoding separately. Re-run EWHA using the measured q_l after routing because the layer reach profile has changed.

Experiment E - sequence mixer selection

Compare Mamba-2, Gated DeltaNet, Preconditioned DeltaNet, and one hybrid attention configuration on MQAR, overwrite, RULER, and language loss. Keep the best operator based on the quality-memory-latency trade-off, not on an asymptotic complexity claim.

First-cycle deliverable

At the end of the first cycle, the research group should produce (i) a frozen per-layer $\alpha_l/\beta_l/\gamma_l$ allocation table, (ii) the corresponding q_l routing histogram, (iii) a quality-versus-cost comparison against fixed baselines, (iv) a packed runtime measurement, and (v) a decision to proceed, revise, or stop. That package is more valuable than a large uninstrumented training run.

20. Conclusion

The revised THME proposal is centered on a more precise thesis: representational heterogeneity should be selected per layer and per channel group, while adaptive-depth routing independently determines which tokens execute those layers. The two mechanisms should be optimized together because their expected costs interact. The 80/20 scenario is a testable routing target; it is not a rule for assigning α, β, and γ.

The proposed Execution-Weighted Heterogeneous Allocation procedure turns this idea into an implementable research question. It estimates layer reach probabilities, measures candidate format sensitivity and hardware cost, solves a discrete allocation problem, fine-tunes only the strongest candidates, recalibrates the router, and freezes a deployable artifact. Its objective is to improve a measured quality-energy-memory-latency frontier, not to maximize mathematical complexity or the number of novel components.

The first experiments should be inexpensive: frozen-model calibration, controlled recall tasks, layerwise perturbation, actual packing, and kernel microbenchmarks. Only mechanisms that survive these tests should receive full model-training compute. A successful THME outcome may be a compact combination of ternary weights, an effective hybrid sequence mixer, learned early-exit routing, and sensitivity-driven allocation. The optional geometric and dynamical mechanisms should earn their place independently.

The programme is designed to produce a credible positive result, a useful partial result, or a defensible negative result without spending large compute on an architecture whose individual assumptions have not yet been tested.

Appendix A. Compact formal specification

A.1 Layerwise representation allocation

For each layer l and group g: choose exactly one family $s \in \{B,T,P\}$.

$z_{lgs} \in \{0,1\}; \quad \Sigma_s z_{lgs} = 1$

$\alpha_l = d_l^{-1} \Sigma_g n_{lg} z_{lgB}; \ \beta_l = d_l^{-1} \Sigma_g n_{lg} z_{lgT}; \ \gamma_l = d_l^{-1} \Sigma_g n_{lg} z_{lgP}$

$\alpha_l + \beta_l + \gamma_l = 1$

A.2 Execution probability and expected resource use

q_l = Pr(token reaches layer l)

$C_expected = \Sigma_l q_l C_l(\alpha_l, \beta_l, \gamma_l; \text{hardware, batch, context})$

$W_model = \Sigma_l W_l + W_embeddings + W_head + W_metadata + W_auxiliary$

$W_traffic/token \approx \Sigma_l q_l W_{l,active}$ (corrected by measured cache/residency behavior)

A.3 Sensitivity-weighted distortion approximation

$\hat{D} = \Sigma_l \Sigma_g \Sigma_s q_l \cdot z_{lgs} \cdot \Omega_{lg} \cdot d_{lgs}$

D-hat is a screening approximation. Ω_lg is a sensitivity weight; d_lgs is the measured or estimated perturbation associated with assigning family s. The full model evaluation is authoritative because interaction terms can violate additivity.

A.4 Primary EWHA optimization

minimize $\Sigma_l \Sigma_g \Sigma_s q_l z_{lgs} E_{lgs}$

subject to $\hat{D} \le \varepsilon_Q$; $model_bytes \le S_max$; $traffic \le R_max$; $latency \le T_max$

$\Sigma_s z_{lgs} = 1; \quad z_{lgs} \in \{0,1\}$

A.5 Joint routing-allocation iteration

16. Initialize router policy π^0 and allocation z^0.
17. Estimate $q_l(\pi^0)$ on the calibration workload.
18. Solve EWHA for z^1 using $q_l(\pi^0)$ and measured cost/sensitivity tables.
19. Fine-tune or distill the model under z^1 and update router π^1.
20. Re-estimate $q_l(\pi^1)$; accept the update only on selection-split improvement.
21. Repeat only up to a prespecified iteration cap; freeze final z and π before the final test.

Appendix B. Illustrative layer allocation table

The table below shows the form of the output expected from EWHA. Values are intentionally illustrative and are not recommended fixed shares. They simply demonstrate that each layer can have a different α/β/γ vector while the 80/20 token reach profile is held constant.

Layer	q_l	α_l binary	β_l ternary	γ_l phase	Status
L1	1.00	0.20	0.75	0.05	Illustrative only
L2	1.00	0.15	0.75	0.10	Illustrative only
L3	1.00	0.12	0.73	0.15	Illustrative only
L4	1.00	0.10	0.68	0.22	Illustrative only
L5	0.20	0.10	0.60	0.30	Illustrative only
L6	0.20	0.08	0.57	0.35	Illustrative only
L7	0.20	0.08	0.50	0.42	Illustrative only

Layer	q_l	α_l binary	β_l ternary	γ_l phase	Status
L8	0.20	0.06	0.48	0.46	Illustrative only
L9	0.20	0.05	0.43	0.52	Illustrative only
L10	0.20	0.05	0.35	0.60	Illustrative only
L11	0.20	0.05	0.30	0.65	Illustrative only
L12	0.20	0.05	0.25	0.70	Illustrative only

The example has the same illustrative q_l profile for all candidate schedules. EWHA is free to return non-monotonic layer distributions if the measured sensitivity/cost evidence supports them. The example must not be used as a prior truth or as a default hand-coded schedule.

Appendix C. Experiment registry template

Field	Required entry
Experiment ID / hypothesis	Unique ID and H1-H11 mapping.
Model / checkpoint	Architecture, parameter count, tokenizer, checkpoint hash.
Dataset	Name, version, split, checksum, contamination check.
Allocation	Per-layer α_l/β_l/γ_l, group size, family definitions, metadata precision.
Routing	Exit layer(s), target p_exit, achieved q_l, router checkpoint and threshold.
Training	Tokens, optimizer, learning rate, batch, sequence length, steps, seeds, FLOPs.
Hardware	CPU/GPU/accelerator, memory, driver, framework, kernel commit.
Measurements	NLL, benchmark scores, bytes, traffic, latency, throughput, joules/token.
Statistics	Primary endpoint, margin, CI method, correction, number of seeds.
Decision	Pass, fail, revise, or pending; reason; next permitted stage.

Appendix D. Minimum reproducibility checklist

- Publish the exact model configuration and all tensor shapes.
- Publish the per-layer α_l, β_l, γ_l fractions and group-to-format indices.
- Publish the measured q_l histogram for each evaluation workload.
- Publish actual serialized bytes for every tensor category.
- Publish quantization scales, codebooks, phase magnitude and phase index encodings.
- Publish kernel source/build configuration and profiler traces.
- Publish training tokens, FLOPs, hardware-hours, energy and memory peaks.
- Publish all seeds and checkpoint-selection criteria.
- Publish raw benchmark predictions and analysis scripts.
- Publish negative results and the reasons components were removed.

Appendix E. Research decision gates

Gate	Proceed only if	Otherwise
0 - Specification	Operator, storage, routing and kernel contracts are explicit.	Clarify or remove undefined operator.
1 - Cheap screening	At least one candidate has a plausible	Stop that branch.

Gate	Proceed only if	Otherwise
	quality-cost gain and a real kernel path.	
2 - Pilot	Matched pilot confirms a promising quality-cost curve.	Revise one variable or stop.
3 - Confirmation	Gain survives seeds, scales, matched baselines and untouched evaluation.	Do not scale.
4 - Systems	Gain survives packed deployment, realistic batches and energy measurement.	Reframe as quality/size result only.
5 - Replication	Independent group or independent implementation reproduces principal finding.	Do not claim a breakthrough.

Appendix F. Terminology and notation

Symbol / term	Definition
L	Total number of sequential layers.
l	Layer index.
d_l	Channel/residual width of layer l.
g	Channel group within a layer.
s ∈ {B,T,P}	Binary, ternary, or phase-oriented family.
α_l	Fraction of layer l channels assigned to the binary family.
β_l	Fraction of layer l channels assigned to the ternary family.
γ_l	Fraction of layer l channels assigned to the phase family.
q_l	Probability that a token executes layer l.
p_exit	Fraction of tokens exiting at a defined early checkpoint.
p_continue	Fraction of tokens proceeding beyond that checkpoint.
z_lgs	Binary allocation variable for group g of layer l to family s.
Ω_lg	Estimated sensitivity of group g in layer l.
ΔL_lgs	Estimated task-loss increase from assigning format s.
EWHA	Execution-Weighted Heterogeneous Allocation, the proposed offline allocator.
MQAR	Multi-query associative recall benchmark.
NLL	Negative log-likelihood / token-level cross-entropy.

References

[1] Ma, S., Wang, H., Huang, S., Zhang, X., Hu, Y., Song, T., Xia, Y., & Wei, F. (2025). BitNet b1.58 2B4T Technical Report. arXiv:2504.12285. https://arxiv.org/abs/2504.12285

[2] Dao, T., & Gu, A. (2024). Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. Proceedings of the 41st International Conference on Machine Learning, PMLR 235, 10041-10071. https://proceedings.mlr.press/v235/dao24a.html

[3] Yang, S., Kautz, J., & Hatamizadeh, A. (2025). Gated Delta Networks: Improving Mamba2 with Delta Rule. International Conference on Learning Representations (ICLR 2025). https://arxiv.org/abs/2412.06464

[4] Tumma, N., Loo, N., & Rus, D. (2026). Preconditioned DeltaNet: Curvature-aware Sequence Modeling for Linear Recurrences. Proceedings of the 43rd International Conference on Machine Learning, PMLR 306, 122566-122605. https://proceedings.mlr.press/v306/tumma26a.html

[5] Arora, S., Eyuboglu, S., Timalsina, A., Johnson, I., Poli, M., Zou, J., Rudra, A., & Ré, C. (2024). Zoology: Measuring and Improving Recall in Efficient Language Models. International Conference on Learning Representations (ICLR 2024). https://arxiv.org/abs/2312.04927

[6] Raposo, D., Ritter, S., Richards, B., Lillicrap, T., Humphreys, P. C., & Santoro, A. (2024). Mixture-of-Depths: Dynamically allocating compute in transformer-based language models. arXiv:2404.02258. https://arxiv.org/abs/2404.02258

[7] Elhoushi, M., Shrivastava, A., Liskovich, D., Hosmer, B., Wasti, B., Lai, L., Mahmoud, A., Acun, B., Agarwal, S., Roman, A., Aly, A., Chen, B., & Wu, C.-J. (2024). LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. Proceedings of ACL 2024, 12622-12642. https://aclanthology.org/2024.acl-long.681/

[8] Wang, K., Liu, Z., Lin, Y., Lin, J., & Han, S. (2019). HAQ: Hardware-Aware Automated Quantization With Mixed Precision. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. https://openaccess.thecvf.com/content_CVPR_2019/html/Wang_HAQ_Hardware-Aware_Automated_Quantization_With_Mixed_Precision_CVPR_2019_paper.html

[9] Yao, Z., Dong, Z., Zheng, Z., Gholami, A., Yu, J., Tan, E., Wang, L., Huang, Q., Wang, Y., Mahoney, M. W., & Keutzer, K. (2021). HAWQ-V3: Dyadic Neural Network Quantization. Proceedings of the 38th International Conference on Machine Learning, PMLR 139, 11875-11886. https://proceedings.mlr.press/v139/yao21a.html

[10] Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., & Han, S. (2023). AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. arXiv:2306.00978. https://arxiv.org/abs/2306.00978

[11] Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., & Han, S. (2023). SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. Proceedings of the 40th International Conference on Machine Learning, PMLR 202, 38087-38099. https://proceedings.mlr.press/v202/xiao23c.html

[12] Hsieh, C.-P., Sun, S., Kriman, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., & Ginsburg, B. (2024). RULER: What's the Real Context Size of Your Long-Context Language Models? arXiv:2404.06654. https://arxiv.org/abs/2404.06654

[13] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems 30. https://arxiv.org/abs/1706.03762

[14] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. Advances in Neural Information Processing Systems 35. https://arxiv.org/abs/2205.14135

[15] Ma, S., Wang, H., Ma, L., Wang, L., Wang, W., Huang, S., Dong, L., Xue, J., & Wei, F. (2024). The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. arXiv:2402.17764. https://arxiv.org/abs/2402.17764

[16] Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the Knowledge in a Neural Network. arXiv:1503.02531. https://arxiv.org/abs/1503.02531